

Prescription Modeling in the FEN Schema: Facts, ProblemEpisodes, and Medication-State Assessment

Phoros Engineering

August 2026

1 Status and Adopted Direction

This document specifies how prescription and medication-order information participates in the Facts–Episodes–Narratives (FEN) ontology. It replaces the earlier comparison among four mutually exclusive lifecycle options.

Adopted direction.

1. A medication order and each subsequent lifecycle event are Facts.
2. Medication Facts may be grouped through authored `EpisodeMembership` records into a medication-management `ProblemEpisode`.
3. The schema does not determine the boundary of that `ProblemEpisode`. The boundary is established by whoever authors the memberships and remains contestable in the ordinary FEN manner.
4. Source medication-order identifiers are external references. They may support an author's membership judgment, but they are not FEN course identifiers and do not determine episode identity.
5. Current medication state is assessed from applicable Facts. A transient assessment is a projection. If Phoros persists the assessment and takes epistemic responsibility for it, the assessment is a Phoros-authored inference Fact whose assertion method is `Inferred` and whose evidence cites its inputs.

No fourth top-level medication entity is introduced. A mutable `MedicationOrder` may still be useful as an interoperability resource, API representation, or materialized read model, but it is not part of the core FEN ontology.

1.1 Reading the source-to-Fact boundary

Suppose an EHR export contains a medication order in a PDF or EHI table. The received file is preserved outside FEN as an immutable `SourceArtifact`, with its bytes held by a `StoredObject`. A `SourceCitationRef` identifies the relevant page, region, table row, or field. The normalized medication event is a Fact: its `assertion:` `AssertionProvenance` records who or what admitted the claim, when, and how; its `evidence:` `Vec<EvidenceRef>` cites the source location or prior Facts that support it. A source order identifier belongs in `external_refs`; it supports correlation, not epistemic justification.

Concern	Representation
Original medication document or export	<code>SourceArtifact</code> $\rightarrow$ <code>StoredObject</code>
Exact supporting passage or row	<code>SourceCitationRef</code>
Who asserted the normalized event and how	<code>AssertionProvenance</code>
Supporting source material or prior Facts	<code>Fact.evidence</code>
Source order, request, version, or event identity	<code>Fact.external_refs</code>
Transient processing or display state	<code>FactOperational</code> or projection

2 Epistemic Commitments

2.1 Facts carry assertion metadata and evidence

A Fact is not guaranteed truth. It is an epistemic record admitted to the system with assertion metadata, evidence, a temporal anchor, and status. Assertion metadata records who or what asserted the Fact, when, and by which method. Evidence records the source citations and prior Facts that support the claim. Medication Facts may be directly observed, authored by a clinician or patient, reported by an institution, extracted from a source document, or inferred by Phoros. These distinctions determine how a Fact should be evaluated; they do not determine whether it is ontologically a Fact.

Examples include:

- a clinician authored a medication order;
- an EHR reported that an order was modified or discontinued;
- a pharmacy reported that medication was dispensed;
- a patient stated that they stopped taking a medication;
- Phoros inferred a current medication state or a possible interaction;
- a clinician authored a Note explaining the reason for a change.

A clinician-authored Note is therefore prime Fact material. Its content may be challenged, contradicted, refined, or interpreted by other epistemic objects, but the event and content of its authorship remain a Fact.

2.2 ProblemEpisodes are authored problem-oriented groupings

A `ProblemEpisode` is used when treating a collection of Facts separately obscures a problem those Facts address together. A medication course can be represented as such a problem-oriented grouping, but the schema does not supply an objective course-boundary algorithm.

An author may judge that two renewals belong to one medication-management `ProblemEpisode`, while another author may distinguish them. One source order may participate in

multiple ProblemEpisodes. Multiple source order identifiers may participate in one ProblemEpisode. A medication Fact may initially have no Episode membership at all.

All such cases are represented by ordinary `EpisodeMembership` assertion metadata, status, and contestability. Membership is not a Fact, so its `asserted_by` and `asserted_at` fields remain distinct from `Fact.assertion`. A source order identifier, shared ingredient code, temporal adjacency, or automated correlation result may support a membership judgment; none is a schema-level rule that creates the boundary. If membership evidence becomes first-class, it must remain distinct from the metadata about who authored the membership.

2.3 Narratives provide higher-order accounts

A **Narrative** is a higher-order authored account of a ProblemEpisode. It is not distinguished from Fact merely because it contains interpretation: authored and inferred assertions may also be Facts. Narrative is distinguished by its episode-level explanatory and synthesizing role.

For example, a clinician’s Note explaining a dose change is a Fact. A later longitudinal account explaining how renal function, adverse effects, and patient preferences shaped several medication decisions is a Narrative.

2.4 Projections are not a fourth ontological layer

Read models, indexes, and transient calculations may be materialized for performance or presentation without becoming new FEN entity types. The distinction is one of epistemic commitment:

- A transient calculation used to render a screen is a projection.
- A persisted assertion for which Phoros records authorship and evidence is an inference Fact with `AssertionMethod::Inferred`.

3 Problems with the Prior Prescription Payload

The prior payload was approximately:

```
1 Prescription {
2   medication:    CodedValue,
3   dosage:        Dosage,
4   frequency:     Frequency,      // previously Vec<Duration>
5   period:        Option<TimeInterval>,
6   relevant_note: Option<FactId>,
7 }
```

It has five separable problems.

3.1 Frequency cannot be represented as `Vec<Duration>`

A sequence of time gaps cannot faithfully express common medication instructions. It cannot adequately distinguish “twice daily” from “every twelve hours,” or represent PRN use,

frequency ranges, day-of-week schedules, times of day, relationships to meals, titration stages, conditional holds, rates, or maximum doses.

Medication instructions must preserve both structured information and the original SIG text. The exact Rust representation can evolve independently of the lifecycle decision, but it must support at least:

- ordered or concurrent instruction stages;
- frequency and frequency ranges over a period;
- days of week, times of day, and event-relative timing;
- PRN indication and other conditions;
- dose, route, method, rate, and maximum-dose constraints;
- intended bounds; and
- original source text and source coding.

A representative internal shape is:

```
1 pub struct MedicationInstruction {
2     pub dosage: Dosage,
3     pub timing: MedicationTiming,
4     pub route: Option<CodedValue>,
5     pub method: Option<CodedValue>,
6     pub as_needed_for: Vec<CodedValue>,
7     pub maximum_dose: Option<MaximumDose>,
8     pub original_sig: Option<String>,
9 }
10
11 pub struct MedicationTiming {
12     pub code: Option<CodedValue>,
13     pub repeat: Option<RepeatPattern>,
14     pub event_offsets: Vec<EventOffset>,
15     pub text: Option<String>,
16 }
```

This is a requirements sketch rather than a final medication-timing specification.

3.2 Temporal fields had overlapping meanings

The former `Fact.occurred_at` and `Prescription.period` fields did not state whether they meant order authorship, intended effectiveness, actual medication use, or the duration of a retrospectively grouped course.

The revised contract is:

- `Fact.occurred_at` records when the epistemic event represented by the `Fact` occurred or was reported to occur.
- `effective_from` records a source-authored prospective effective date when it differs from the order event.
- `intended_duration` records the stated prospective intent.
- A stop, hold, resume, or modification is represented by another `Fact`.

- A ProblemEpisode does not acquire a schema-determined interval merely because its medication Facts can be temporally ordered.

An inferred actual duration may be calculated from selected Facts and memberships, but it remains an assessment whose result can be ambiguous.

3.3 The lifecycle event was implicit

The former payload could show two different medication snapshots but could not state whether the difference represented an initial order, correction, dose modification, hold, renewal, or discontinuation. Comparing values is not a substitute for preserving the event asserted by the source.

3.4 Fact status was easily confused with medication state

A later dose change does not make the earlier order cease to have occurred. Likewise, discontinuation does not make prior order and modification Facts epistemically inactive. Medication lifecycle state and `FactStatus` answer different questions and must not share one state machine.

3.5 One relevant Note reference was too narrow

A medication assertion may be supported by Notes, laboratory results, diagnoses, dispensing records, or Facts from another domain. The singular `relevant_note` field is therefore removed. General epistemic support belongs in the enclosing Fact's evidence collection. Prior Facts use the `PriorFact` evidence variant; source passages use `Source`. A referenced Fact retains its own authorization policy; the reference does not grant access to its target.

4 Adopted Medication Event Fact Model

The adopted design combines the best properties of the earlier event-stream and order-plus-delta proposals. Every lifecycle event is a Fact. An initial order carries a complete instruction. A modification records what the source says changed and, when the source supplies it, the complete resulting instruction.

4.1 Payload family

Medication payloads belong to the closed clinical Fact payload family. The shared FEN ontology does not require one global enum containing identity, clinical, and health-economic variants.

```

1 pub enum ClinicalFactPayload {
2     // Other closed clinical variants...
3     MedicationOrderEvent(MedicationOrderEventPayload),
4     MedicationCourseAssessment(MedicationCourseAssessmentPayload),
5     // The interaction representation remains deferred.
6 }
7
8 pub struct MedicationOrderEventPayload {

```

```

9     pub medication: CodedValue,
10    pub event: MedicationOrderEvent,
11 }

```

The enclosing Fact supplies the common identity, subject, temporal, status, assertion, evidence, and external-reference fields. Payload-level Fact IDs are retained only where they express medication-specific domain semantics, such as the event targeted by a modification.

4.2 Lifecycle events

```

1  pub enum MedicationOrderEvent {
2      Authored {
3          instruction: MedicationInstruction,
4          intended_duration: Option<Duration>,
5          effective_from: Option<ApproximateDate>,
6      },
7
8      Modified {
9          target_fact_id: Option<FactId>,
10         changes: MedicationInstructionPatch,
11         resulting_instruction: Option<MedicationInstruction>,
12         reason: Option<String>,
13     },
14
15     Held {
16         target_fact_id: Option<FactId>,
17         reason: Option<String>,
18         expected_resume: Option<ApproximateDate>,
19     },
20
21     Resumed {
22         target_fact_id: Option<FactId>,
23     },
24
25     Discontinued {
26         target_fact_id: Option<FactId>,
27         reason: Option<String>,
28     },
29
30     Renewed {
31         prior_order_fact_id: Option<FactId>,
32         instruction: MedicationInstruction,
33         intended_duration: Option<Duration>,
34     },
35
36     ReportedSnapshot {
37         instruction: MedicationInstruction,
38         reported_status: Option<ReportedMedicationStatus>,
39         effective_from: Option<ApproximateDate>,
40         effective_through: Option<ApproximateDate>,
41     },
42 }

```

`ReportedSnapshot` is retained for sources and migrations that provide a medication state without reliable evidence of the lifecycle event that produced it. Ingestion must not invent `Authored` or `Modified` semantics merely to fit a preferred event model.

4.3 Patch semantics

A modification must distinguish an omitted field from a field explicitly removed by the source.

```
1 pub struct MedicationInstructionPatch {
2     pub dosage: Option<FieldChange<Dosage>>,
3     pub timing: Option<FieldChange<MedicationTiming>>,
4     pub route: Option<FieldChange<CodedValue>>,
5     pub method: Option<FieldChange<CodedValue>>,
6     pub as_needed_for: Option<FieldChange<Vec<CodedValue>>>,
7     pub maximum_dose: Option<FieldChange<MaximumDose>>,
8     pub original_sig: Option<FieldChange<String>>,
9 }
10
11 pub enum FieldChange<T> {
12     Set(T),
13     Remove,
14 }
```

The two fields on `Modified` make different epistemic commitments:

- `changes` records what the source identified as changing.
- `resulting_instruction` records a complete post-change instruction only when the source explicitly supplies one.

Phoros must not interpret an omitted field as a source-authored assertion that the field remained unchanged unless the source contract establishes that semantics.

4.4 Source identifiers

The model introduces no FEN `MedicationCourseId` or internal `OrderGroupId`. Source order, request, version, and event identifiers are recorded through `Fact.external_refs`. The `resource_type` or equivalent discriminator distinguishes an order identifier from an event or version identifier.

These identifiers answer “which source record is this?” They do not answer “what supports this assertion?” The latter requires a `SourceCitationRef` to the relevant source passage, row, or field in `Fact.evidence`.

A `target_fact_id` is useful when a source or author states that one event specifically changes another. It is not a substitute for `ProblemEpisode` membership and is not required when the target cannot be established.

5 ProblemEpisode Membership

Medication Facts participate in the existing FEN membership mechanism:

```

1 EpisodeMembership {
2   id: MembershipId("mem-2"),
3   fact_id: FactId("f-metformin-modified"),
4   episode_id: ProblemEpisodeId("ep-metformin-management"),
5   role: FactRole::Treatment,
6   asserted_by: Author::clinician("Dr. Patel"),
7   asserted_at: TemporalAnchor::Point(Timestamp("2026-01-16")),
8   status: MembershipStatus::Active,
9 }

```

The schema deliberately does not answer whether a renewal, restart, changed indication, formulation change, or long treatment gap begins a new ProblemEpisode. That is a question for the membership author. Automated systems may author memberships, but they must do so with identifiable authorship and the same contestability expected of other membership assertions. Automated authoring also records the rule or model version in the appropriate audit or operational record; this mechanism metadata must not be collapsed into membership authorship.

A medication Fact may also belong to a broader clinical ProblemEpisode. For example, the same dose-modification Fact may be a member of both a metformin management ProblemEpisode and a chronic kidney disease ProblemEpisode, with a different FactRole in each.

6 Fact Status and Medication Lifecycle State

FactStatus expresses the system's current disposition toward an epistemic record:

```

1 pub enum FactStatus {
2   Active,
3   Superseded {
4     replaced_by: Option<FactId>,
5     reason: SupersessionReason,
6     // author and time omitted here for brevity
7   },
8   EnteredInError {
9     replaced_by: Option<FactId>,
10    // author and time omitted here for brevity
11  },
12 }

```

Medication lifecycle state expresses what the available evidence says about the medication:

```

1 pub enum MedicationLifecycleState {
2   Ordered,
3   Active,
4   OnHold,
5   Discontinued,
6   Completed,
7   Unknown,
8 }

```

These states must not be conflated. Consider:

Fact	Fact status	Course effect
Metformin 500 mg ordered on January 1	Active	Establishes order
Dose increased to 1000 mg on March 1	Active	Changes dose
Medication discontinued on June 1	Active	Ends active use

All three Facts remain valid records of events that occurred even though the assessed medication state is **Discontinued**.

Supersession is appropriate when a new Fact replaces the epistemic position of an earlier Fact. For example, an imported Fact saying that the original order was 500 mg may be superseded by a corrected Fact grounded in authoritative source evidence that establishes it was 250 mg. Ordinary dose changes, holds, renewals, and discontinuations are not supersessions.

Fact.status may store the materialized current disposition. The history of status transitions must nevertheless remain append-only and reconstructable so the system can answer both “what is the Fact’s status now?” and “what did the system regard as active at time t ?” The storage mechanism for that history is outside this prescription specification.

7 Medication-Course State Assessment

The earlier term **FoldMedicationState** implied a complete, consistent, totally ordered event stream. FEN cannot assume those conditions. Relevant Facts may conflict, come from different sources, have uncertain membership, or support more than one present-state interpretation.

The evaluator is therefore described as an assessment:

```

1 pub enum MedicationCourseAssessment {
2   Determinate {
3     state: CurrentMedicationState,
4   },
5
6   Ambiguous {
7     candidate_states: Vec<CurrentMedicationState>,
8     conflicting_facts: Vec<FactId>,
9   },
10
11   InsufficientEvidence {
12     relevant_facts: Vec<FactId>,
13   },
14 }

```

An assessment request identifies a **ProblemEpisode**, an **as_of** time, and an authorized view of its memberships and Facts. The evaluator must record the rule or model version and all input Fact IDs on which its result depends.

If the result is transient, it remains a projection. If Phoros persists the result as an epistemic

assertion, it is represented as a Phoros-authored inference Fact:

```
1 pub struct MedicationCourseAssessmentPayload {
2     pub problem_episode_id: ProblemEpisodeId,
3     pub as_of: Timestamp,
4     pub assessment: MedicationCourseAssessment,
5     pub evaluator: RuleArtifactRef,
6 }
```

The enclosing inference Fact records `AssertionMethod::Inferred` and cites every epistemic input through `Fact.evidence`. The `conflicting_facts` and `relevant_facts` fields remain inside the assessment because they explain why the result is ambiguous or insufficient; when the assessment is persisted, those same Fact IDs must also appear as evidence. This deliberate duplication is validated for consistency.

Persisted assessments may be superseded when new evidence arrives or the evaluator changes. That is epistemic replacement and is appropriately represented through `FactStatus` and `SupersessionReason::RuleReEvaluation`.

8 Notes, Evidence, and Inferred Rationales

An event payload may carry a reason when that reason was part of the source assertion. A clinician-authored Note that explains the decision is itself a Fact and may be cited through `Fact.evidence`. The medication event's source citation establishes what the clinician actually authored.

Phoros must not insert an inferred rationale into a clinician-authored Fact as though the clinician supplied it. Instead, Phoros authors a separate inference Fact:

```
1 pub struct MedicationRationaleInferencePayload {
2     pub medication_event_fact_id: FactId,
3     pub rationale: String,
4     pub confidence: Option<Confidence>,
5     pub evaluator: RuleArtifactRef,
6 }
```

This permits a Phoros inference to cite the medication event, relevant Note, laboratory results, and Facts from other domains without changing the authorship of any original Fact. The payload's `medication_event_fact_id` identifies the assertion being explained; the enclosing inference Fact's `evidence` records the inputs that support the inferred rationale.

9 Medication Interactions

Medication “interaction” covers several distinct claims:

1. contraindication or conflict;
2. counter-medication used to manage an effect of another medication;
3. intentional joint regimen;
4. dose-modifying interaction;

5. causal chain in which one medication creates a need for another; and
6. sequential replacement or supersession for a stated reason.

General drug–drug knowledge is contextual reference data, not a patient Fact. When a clinician, institution, knowledge system, or Phoros asserts that an interaction applies to a particular subject, that patient-specific assertion may be represented as a Fact:

```

1 pub struct MedicationInteractionAssertionPayload {
2     pub involved_facts: Vec<FactId>,
3     pub interaction_type: MedicationInteractionType,
4     pub assertion: Option<String>,
5     pub knowledge_source: Option<ReferenceDataRef>,
6     pub evaluator: Option<RuleArtifactRef>,
7     pub confidence: Option<Confidence>,
8 }
9
10 pub enum MedicationInteractionType {
11     Contraindication,
12     CounterMedication,
13     JointRegimen,
14     DoseModifying,
15     CausedNeedFor,
16     Replaced,
17 }

```

The enclosing Fact records assertion metadata and evidence. The `involved_facts` field identifies the medications participating in the interaction; it is domain content rather than a substitute for evidence. An inferred interaction cites its patient-specific input Facts and the relevant location in the versioned knowledge artifact. `ReferenceDataRef` identifies the knowledge version, while a source citation identifies the supporting passage; `evaluator` identifies the rule or model that applied it.

Other compatible representations remain available for different purposes:

- A typed relation between ProblemEpisodes may express a relationship between two medication-management problems.
- `EpisodeMembership.role` may state the function a Fact plays inside one ProblemEpisode, but cannot by itself represent a complete pairwise interaction.
- A Narrative or DecisionPoint may explain how an interaction affected treatment reasoning.

This document does not yet choose one authoritative representation for every interaction use case. The adopted medication lifecycle model is compatible with all three and does not require a fourth medication entity.

10 Rejected Alternatives

10.1 Using Fact supersession for clinical evolution

Rejected. A dose modification, hold, resume, renewal, or discontinuation is a new epistemic event, not a correction of the earlier event. Reusing `FactStatus::Superseded` would erase

the distinction between change in the world and replacement of an epistemic record.

10.2 A mandatory internal medication-order grouping identifier

Rejected as the FEN course identity. Source order identifiers remain external references, and ProblemEpisode membership supplies authored grouping. An operational correlation key may exist in an ingestion or read-model layer, but it does not determine ProblemEpisode boundaries.

10.3 A separate top-level MedicationOrder entity

Rejected as a core FEN entity. The proposed mutable object combined event history, current state, and grouping into a fourth semantic layer. Its useful features are already covered by medication event Facts, ProblemEpisodes, and state assessments. A derived MedicationOrder representation remains permissible for FHIR interchange, APIs, or optimized reads.

10.4 Full snapshots only

Rejected as the sole lifecycle representation. Full snapshots are retained when that is all a source supplies, but they must be labeled as reported snapshots. When the source explicitly reports a modification, the Fact should preserve the modification semantics rather than requiring consumers to infer them by comparing snapshots.

11 Scope Beyond Prescription

Coverage, prior authorization, diagnosis certainty, and other longitudinal domains may exhibit a similar pattern: multiple Facts become intelligible only when considered under a ProblemEpisode. Their specific payloads, membership practices, and state assessments belong to their respective domain specifications. This prescription document makes no binding lifecycle decision for them.

12 Migration Considerations

Existing prescription records must be migrated without inventing epistemic specificity that the source does not contain.

Migration begins by registering the original document, archive, or export as a **SourceArtifact** and preserving its bytes in a **StoredObject**. The migration then creates citations to the relevant medication-list row, order, or Note before constructing normalized Facts.

1. An existing Fact with evidence of an initial authored order maps to the **Authored** event variant.
2. An existing source snapshot whose lifecycle event cannot be established maps to **ReportedSnapshot**.
3. Existing source order identifiers migrate to **Fact.external_refs**; they do not become ProblemEpisode IDs.

4. The old relevant-Note value becomes a **PriorFact** entry in **Fact.evidence**.
5. Existing Facts are not assigned to ProblemEpisodes by schema migration alone. A separate authored or automated membership process may propose memberships with explicit authorship and audit metadata.
6. Existing Facts are not superseded merely because a later medication record exists. Supersession is preserved only where the migration can establish genuine epistemic replacement or correction.
7. A migration-generated resulting instruction is an inferred assertion. It cites the source material and prior Facts used to build it and records the reconstruction rule version.

This ordering ensures that normalization never becomes the sole surviving representation of the source and that future parsers can revisit the original meaning.

13 Decision Summary

Concern	FEN representation
Initial medication order	Fact
Modification, hold, resume, stop, renewal	Facts
Original medication document or export	SourceArtifact outside FEN
Exact supporting passage, row, or field	SourceCitationRef
Supporting Note, lab, or prior medication Fact	EvidenceRef::PriorFact
Source medication-order identity	ExternalRef
Medication-management grouping	ProblemEpisode plus authored EpisodeMembership
ProblemEpisode boundary	Determined by membership authors, not by schema
Specific event-to-event dependency	Direct FactId or typed Fact relation
Present-state calculation	Projection or Phoros-authored inference Fact
Clinician Note and stated rationale	Fact
Phoros-inferred rationale	Separate inference Fact
Correction of an epistemic record	FactStatus::Superseded or EnteredInError
Medication no longer active	Medication lifecycle assessment, not FactStatus
Mutable interoperability/API resource	Derived or materialized representation outside core FEN
Transient processing or indexing state	FactOperational or projection

The central commitment is that prescription modeling uses the existing FEN ontology without collapsing clinical evolution into Fact replacement and without introducing a fourth medication layer. Facts preserve what was authored, observed, reported, or inferred. ProblemEpisodes group those Facts under author-defined problems. Narratives synthesize their meaning. Read models and transient assessments remain operational artifacts unless Phoros persists them as inference Facts with explicit assertion metadata and evidence.